

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №17

Тема: Основи створення алгоритмів

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Ознайомитися з поняттям алгоритму та його основними властивостями, способами запису алгоритмів (словесний опис, блок-схема, псевдокод, програма), навчитися складати таблицю трасування та розробляти алгоритм поетапно — від постановки задачі до тестування — з подальшою реалізацією мовою Python.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Поняття алгоритму. Алгоритм — це скінченна послідовність чітко визначених дій, виконання яких за скінченну кількість кроків перетворює вхідні дані на шуканий результат. Виконавцем алгоритму може бути людина, автомат або комп'ютер; саме тому записувати алгоритм потрібно так, щоб виконавець не мав потреби щось домислювати.

2. Властивості алгоритму:

- **дискретність** — процес розв'язання розбивається на окремі елементарні кроки (команди);
- **детермінованість** (визначеність) — кожен крок однозначно визначений, а за однакових вхідних даних алгоритм завжди дає однаковий результат;
- **скінченність** (елементарність кроків і завершуваність) — алгоритм завершується за скінченну кількість кроків;
- **результативність** — після завершення алгоритм дає результат або повідомляє, що розв'язку не існує (наприклад, при діленні на нуль);
- **масовість** — алгоритм придатний для цілого класу однотипних задач із різними вхідними даними, а не для єдиного окремого значення.

3. Способи запису алгоритмів.

- **Словесний** — опис кроків природною мовою з нумерацією (простий, але менш точний);
- **Графічний (блок-схема)** — зображення кроків стандартними фігурами, з'єднаними стрілками;
- **Псевдокод** — напівформальний запис, близький до мови програмування, але без суворого синтаксису;
- **Програма** — запис алгоритму мовою програмування, зрозумілою комп'ютеру.

4. Основні елементи блок-схем (за ДСТУ ISO 5807 / ГОСТ 19.701-90):

Фігура	Призначення
Овал (закруглений прямокутник)	Початок і кінець алгоритму
Паралелограм	Введення вхідних і виведення вихідних даних

Фігура	Призначення
Прямокутник	Обчислення, присвоєння значень (процес)
Ромб	Перевірка умови (розгалуження), два виходи — «так» і «ні»
Шестикутник	Заголовок циклу з параметром
Стрілки (лінії зв'язку)	Вказують послідовність виконання кроків

5. Базові алгоритмічні структури. Будь-який алгоритм можна побудувати з трьох базових структур: **слідування** (команди виконуються одна за одною), **розгалуження** (вибір напрямку залежно від умови) та **цикл** (повторення дій, поки виконується умова). Їм присвячено наступні практичні роботи цієї теми.

6. Етапи розроблення алгоритму:

1. Постановка задачі: що дано і що потрібно знайти.
2. Визначення вхідних даних, вихідних даних та проміжних величин (змінних), їхніх типів.
3. Вибір методу розв'язання та запис алгоритму (словесно, блок-схемою, псевдокодом).
4. **Трасування** — ручне виконання алгоритму на конкретних тестових даних із фіксацією значень змінних.
5. Реалізація мовою програмування та тестування на наборі тестів (типові й граничні випадки).

7. Таблиця трасування — таблиця, у якій кожен рядок відповідає одному кроку алгоритму, а стовпці — змінним та умовам. Вона дозволяє побачити, як змінюються значення змінних, знайти помилку та з'ясувати, що саме обчислює невідомий алгоритм.

Хід роботи

Завдання 1. Запис алгоритму різними способами

Задача: знайти більше з двох заданих чисел a і b . Записати алгоритм словесно, у вигляді псевдокоду та блок-схеми, виконати його вручну для кількох наборів даних.

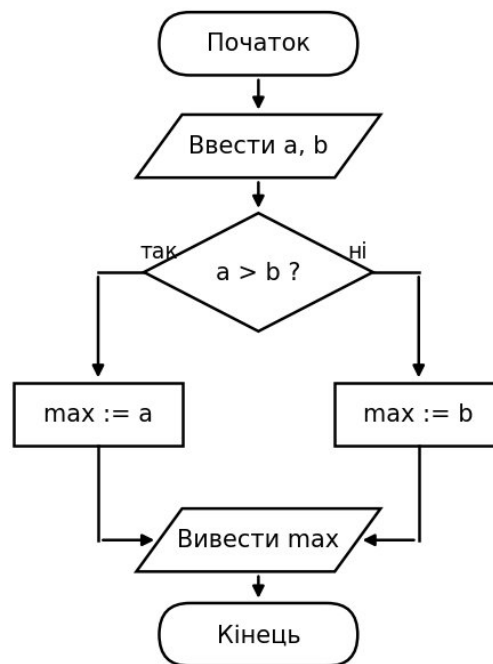
Порядок виконання:

1. Запишіть постановку задачі: вхідні дані (a , b), вихідні дані ($\max$), тип величин (дійсні числа).
2. Складіть словесний опис алгоритму з нумерацією кроків (1. Ввести a і b . 2. Якщо $a > b$, то ... інакше ...).
3. Запишіть той самий алгоритм у вигляді псевдокоду за зразком, наведеним нижче.
4. Накресліть блок-схему алгоритму, використовуючи фігури з таблиці (початок, введення, умова, присвоєння, виведення, кінець); порівняйте її з наведеним зразком.
5. Виконайте алгоритм вручну для пар $(7, 3)$, $(-2, 5)$ та $(4, 4)$; запишіть результат і поясніть, яка гілка алгоритму спрацювала в третьому випадку.

Зразок псевдокоду:

```
алг Більше_з_двох
  ввести a, b
  якщо a > b
    то max := a
    інакше max := b
  все
  вивести max
кін
```

Зразок блок-схеми алгоритму:



Завдання 2. Трасування алгоритму

Дано алгоритм, призначення якого невідоме. Виконати його трасування для $n = 5$ і з'ясувати, що він обчислює.

```
алг Невідомий
  ввести n
  s := 0
  k := 1
  поки k <= n
    s := s + k * k
    k := k + 1
  все
  вивести s
кін
```

Порядок виконання:

1. Побудуйте таблицю трасування зі стовпцями «крок», «умова $k \leq n$ », « k », « s ».
2. Запишіть початковий стан: $n = 5$, $s = 0$, $k = 1$.
3. На кожному кроці перевіряйте умову $k \leq n$; якщо вона істинна — виконайте обидва присвоєння й запишіть нові значення k та s у наступний рядок таблиці.
4. Зафіксуйте крок, на якому умова стає хибною, і значення, яке буде виведено.
5. Сформулюйте словами, що обчислює алгоритм, і запишіть відповідну математичну формулу.
6. Повторіть трасування для $n = 3$ та $n = 0$; поясніть результат для $n = 0$ (скільки разів виконається тіло циклу).

Завдання 3. Програмна реалізація алгоритмів і тестування

Реалізувати мовою Python алгоритми із завдань 1 та 2 у вигляді функцій, перевірити їх на наборі тестів і переконатися, що результати збігаються з ручним трасуванням.

Порядок виконання:

1. Напишіть функцію `max_of_two(a, b)`, що реалізує розгалуження `if ... else` за псевдокодом із завдання 1.
2. Напишіть функцію `sum_of_squares(n)`, що реалізує цикл `while` із завдання 2 (змінні `s`, `k` — як в алгоритмі).
3. Складіть список тестів у вигляді кортежів (вхідні дані, очікуваний результат); включіть типові й граничні випадки (рівні числа, від'ємні числа, $n = 0$, $n = 1$).
4. Виконайте всі тести в циклі й виведіть для кожного «ОК» або «ПОМИЛКА»; порівняйте результат для $n = 5$ із ручним трасуванням.
5. Додатково перевірте функцію `sum_of_squares` за формулою $n(n+1)(2n+1)/6$ для n від 0 до 100.

Приклад реалізації мовою Python:

python

```
def max_of_two(a, b):
    if a > b:
        result = a
    else:
        result = b
    return result

def sum_of_squares(n):
    s = 0
    k = 1
    while k <= n:
        s = s + k * k
        k = k + 1
    return s

tests_max = [(7, 3), 7), ((-2, 5), 5), ((4, 4), 4), ((-8, -3), -3)]
for (a, b), expected in tests_max:
    status = "OK" if max_of_two(a, b) == expected else "ПОМИЛКА"
    print(f"max_of_two({a}, {b}) = {max_of_two(a, b)} {status}")

tests_sq = [(5, 55), (3, 14), (1, 1), (0, 0)]
for n, expected in tests_sq:
    status = "OK" if sum_of_squares(n) == expected else "ПОМИЛКА"
    print(f"sum_of_squares({n}) = {sum_of_squares(n)} {status}")

ok = all(sum_of_squares(n) == n * (n + 1) * (2 * n + 1) // 6 for n in range(101))
print("Перевірка за формулою для n = 0..100:", "успішно" if ok else "є розбіжності")
```

Результат виконання програми:

```
max_of_two(7, 3) = 7   ОК
max_of_two(-2, 5) = 5  ОК
max_of_two(4, 4) = 4   ОК
max_of_two(-8, -3) = -3 ОК
sum_of_squares(5) = 55 ОК
sum_of_squares(3) = 14 ОК
sum_of_squares(1) = 1  ОК
sum_of_squares(0) = 0  ОК
Перевірка за формулою для n = 0..100: успішно
```

Висновок за результатами: усі тести пройдені, а результат для $n = 5$ (55) збігається з ручним трасуванням із завдання 2. Збіг значень функції з формулою $n(n+1)(2n+1)/6$ для всіх n від 0 до 100 підтверджує, що алгоритм обчислює суму квадратів перших n натуральних чисел. Граничні тести (рівні числа, $n = 0$) варто включати обов'язково — саме на них найчастіше виявляються помилки в умовах і межах циклів.

Завдання 4. Перевірка властивостей алгоритму

Дослідити, які властивості алгоритму порушуються в наведених «алгоритмах», і реалізувати захист від порушення властивостей скінченності та результативності.

Порядок виконання:

1. Для кожного з трьох «алгоритмів» визначте порушену властивість і запропонуйте виправлення: (А) «Візьміть трохи борошна й додайте води, скільки потрібно»; (Б) $x := 0$; поки $x \neq 10$: $x := x + 3$; (В) ввести a, b ; вивести a / b .
2. Реалізуйте алгоритм (Б) із лічильником кроків і обмеженням `max_steps`: якщо за `max_steps` кроків умова завершення не виконалась — повідомте про порушення скінченності.
3. Перевірте алгоритм (Б) для початкових значень $x = 1$ (буде досягнуто 10) та $x = 0$ (10 ніколи не буде досягнуто).
4. Реалізуйте алгоритм (В) як функцію `safe_divide(a, b)`, що повертає повідомлення про неможливість обчислення замість аварійного завершення при $b = 0$.
5. Реалізуйте функцію `count_digits(n)`, яка підраховує кількість цифр цілого числа, коректно обробляючи $n = 0$ та від'ємні числа, і продемонструйте її масовість на кількох значеннях.

Приклад реалізації мовою Python:

python

```
def reach_ten(x, step=3, max_steps=1000):
    steps = 0
    while x != 10:
        x = x + step
        steps += 1
        if steps >= max_steps:
            return None, steps
    return x, steps

def safe_divide(a, b):
    if b == 0:
        return "ділення на нуль: результату не існує"
    return a / b

def count_digits(n):
    n = abs(n)
    if n == 0:
        return 1
    count = 0
    while n > 0:
        n = n // 10
        count += 1
    return count

for start in (1, 0):
    res, steps = reach_ten(start)
    if res is None:
        print(f"x0 = {start}: порушено скінченність — за {steps} кроків результату немає")
    else:
        print(f"x0 = {start}: досягнуто {res} за {steps} кроків")

print("safe_divide(9, 4) =", safe_divide(9, 4))
print("safe_divide(9, 0) =", safe_divide(9, 0))
```

```
for n in (0, 7, -45, 1000, 123456789):  
    print(f"count_digits({n}) = {count_digits(n)}")
```

Результат виконання програми:

```
x0 = 1: досягнуто 10 за 3 кроків  
x0 = 0: порушено скінченність — за 1000 кроків результату немає  
safe_divide(9, 4) = 2.25  
safe_divide(9, 0) = ділення на нуль: результату не існує  
count_digits(0) = 1  
count_digits(7) = 1  
count_digits(-45) = 2  
count_digits(1000) = 4  
count_digits(123456789) = 9
```

*Висновок за результатами: для $x_0 = 1$ алгоритм (Б) завершується за 3 кроки, а для $x_0 = 0$ значення 10 ніколи не досягається (0, 3, 6, 9, 12, ...), тому без обмеження кількості кроків цикл був би нескінченним — це пряме порушення властивості **скінченності**. Перевірка $b = 0$ у `safe_divide` забезпечує **результативність**: алгоритм завжди завершується або значенням, або чітким повідомленням. Функція `count_digits` працює для будь-якого цілого числа, що ілюструє властивість **масовості**.*

Контрольні питання

1. Дайте визначення алгоритму. Чому виконавцю не повинно бути потрібно «щось домислювати»?
2. Назвіть і поясніть основні властивості алгоритму.
3. Які способи запису алгоритмів ви знаєте? Які переваги й недоліки має кожен із них?
4. Яке призначення мають фігури блок-схеми: овал, паралелограм, прямокутник, ромб?
5. З яких етапів складається розроблення алгоритму?
6. Що таке трасування алгоритму і яку користь воно приносить?
7. Які три базові алгоритмічні структури використовуються для побудови будь-якого алгоритму?
8. Чому для тестування алгоритму обов'язково потрібно перевіряти граничні випадки?

Зміст звіту

1. Тема, мета роботи.
2. Словесний опис, псевдокод і блок-схема алгоритму (завдання 1) та таблиця трасування (завдання 2).
3. Аналіз порушених властивостей алгоритмів (завдання 4).
4. Тексти програм та результати їх роботи (завдання 3, 4).
5. Висновки.